

# Plan SQA — Power Strike

Trabajo de Campo · Ingeniería de Software II · Grupo 2 Versión: Hito 5 (cierre) — actualizado 06/06/2026. (Base: Plan SQA del Hito 3, actualizado en cada hito.)

## 1. Datos del proyecto

- Proyecto: Power Strike — sistema de gestión de gimnasio.
- Repositorio: <https://github.com/MartinMousist/power-strike>
- Stack: Java 17 + Spring Boot 3.2 (backend) · Vue 3 + Vite + Pinia (frontend) · PostgreSQL · Docker.

## 2. Propósito del plan

Asegurar la calidad del sistema Power Strike garantizando un software funcional, mantenible y verificable. Se busca reducir errores mediante revisiones de código, testing continuo y métricas de calidad, manteniendo trazabilidad entre requerimientos, implementación y pruebas, y mitigando riesgos de fallos funcionales y baja cobertura.

## 3. Roles del equipo

| Integrante      | Rol                         |
|-----------------|-----------------------------|
| Mateo Brizuela  | QA / Desarrollo             |
| Ernesto Ponce   | QC (testing exploratorio)   |
| Martín Mousist  | QC (máquina de estados, CI) |
| Fabrizio Posada | Documentación / FMEA        |

## 4. Estándares y convenciones

- Commits: Conventional Commits (feat, fix, test, docs, chore).
- Trazabilidad: cada test referencia su caso de diseño (CP-, AC-, AT-, EX-); ver docs/hito5/RTM.md.
- Documentación inline: clases y métodos no triviales documentados; TODOs con autor y fecha.
- Estilo: Checkstyle (Java) y ESLint (Vue/TS); sin secretos hardcodeados.

## 5. Estrategia de revisiones (checklist antes de aprobar un PR)

- El código compila / corre sin errores
- El linter no reporta errores críticos
- Los tests pasan (CI en verde)
- Hay tests para el código nuevo
- Nombres claros, sin código comentado sin razón, sin secretos hardcodeados
- Trazabilidad actualizada
- No se mergea con issues críticos abiertos

## 6. Métricas a monitorear y umbrales (valores finales)

| Métrica | Umbral | Valor final | ¿Cumple? |
|---------|--------|-------------|----------|
|---------|--------|-------------|----------|

|                                     |                 |                       |    |
|-------------------------------------|-----------------|-----------------------|----|
| Cobertura en módulos principales    | ≥ 60 %          | 100 %                 | Sí |
| Cobertura global                    | informativa     | 53,0 %                | —  |
| Complejidad ciclomática (CC)        | ≤ 10 por método | 0 violaciones (máx 3) | Sí |
| Maintainability Index (MI)          | alto            | ≈ 85 (estimado)       | Sí |
| Tests automatizados en verde        | 100 %           | 77/77                 | Sí |
| Defectos críticos abiertos al merge | 0               | 0                     | Sí |

Detalle: [docs/hito5/metricas-finales.md](#).

## 7. Herramientas

| Propósito                 | Herramienta                        |
|---------------------------|------------------------------------|
| Testing                   | JUnit 5 + Mockito + Spring MockMvc |
| Cobertura                 | JaCoCo 0.8.11                      |
| Análisis estático (Java)  | Checkstyle 10.x · PMD 6.55.0       |
| Análisis estático (Front) | ESLint                             |
| CI/CD                     | GitHub Actions                     |
| Gestión de defectos       | GitHub Issues                      |

## 8. Frecuencia de medición

- **En cada push / PR:** tests + cobertura + linters (automático, GitHub Actions).
- **Por hito:** revisión de métricas (LOC, CC, MI, cobertura, densidad de defectos) y actualización de este plan.

## 9. Gestión de defectos

- Todo defecto se carga como Issue con título, descripción, pasos, severidad (QA) y prioridad (PO).
- Severidades: Crítico / Mayor / Menor / Cosmético.
- Antes de cerrar un issue se escribe el test que verifica el fix; no se mergea con críticos abiertos.
- Estado actual: 12 defectos detectados, 9 cerrados, 3 abiertos (ver [docs/hito5/plan-pruebas-defectos.md](#)).

## 10. Criterios de aceptación para la entrega final (Hito 5)

- Repositorio público que levanta sin errores + CI en verde.
- Cobertura ≥ 60 % en módulos principales.
- Suite de tests 100 % en verde.
- RTM, métricas finales y reporte de defectos documentados en el repo.
- Análisis heurístico (10 heurísticas de Nielsen).
- Wireframes/mockups en Figma (3 pantallas) — pendiente.

- Defensa oral ensayada — pendiente.

## 11. Riesgos y mitigaciones

| Riesgo                                           | Prob. | Impacto | Mitigación                                                  |
|--------------------------------------------------|-------|---------|-------------------------------------------------------------|
| Defecto crítico de RBAC (DEF-EXP-04)             | —     | Alto    | Corregido en Hito 5 con @PreAuthorize y test de seguridad   |
| Baja cobertura en seguridad/frontend             | Alta  | Medio   | Documentar el alcance; planificar tests en próximos avances |
| Disponibilidad del equipo en semana de entrega   | Media | Medio   | Repartir secciones de la defensa; ensayo previo             |
| Dependencia de un solo integrante para el deploy | Baja  | Medio   | Documentar el docker compose up en el README                |

## 12. Cronograma de calidad

| Fecha            | Actividad                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------|
| Hito 3 (20/05)   | Plan SQA + métricas iniciales                                                                       |
| Hito 4 (03/06)   | Plan de pruebas, 72 tests, CI, FMEA, reporte de defectos                                            |
| Sem 6 (06–16/06) | RTM, métricas finales, heurístico, corrección de defectos (77 tests), wireframes, ensayo de defensa |
| 16/06 (19:00)    | Entrega final + defensa oral                                                                        |